École Nationale Supérieure Polytechnique de Yaoundé

Université de Yaoundé I

PROJET ÉVALUÉ — APPLICATION DÉPLOYÉE SUR AWS

Description de l'API

API HTTP (OpenAPI) et protocole temps réel (WebSocket /collab)

Éditeur de documents partagé pour le journal d'une association étudiante

26 octobre 2026

Sommaire

Serveurs

Authentification

Récapitulatif des routes

Santé

Session et crédit

Documents

Liens et téléchargement

Tableau de bord (admin)

Schémas communs

1. En bref

2. Règles à connaître

3. Refus

4. Messages *stateless* (JSON)

5. Exemple de client minimal (Node 20)

6. Clients et tests fournis

Version 0.8.0 — source : `docs/openapi.yaml` (OpenAPI 3.0.3).

API HTTP de l'éditeur partagé. Toutes les routes sont servies derrière CloudFront (`https://<distribution>.cloudfront.net`), en HTTPS uniquement.

Identité des rédacteurs. Pas de vrais comptes : `POST /api/session` avec un nom et un e-mail renvoie un jeton signé (HMAC-SHA256, valable 24 h), à passer ensuite dans l'en-tête `Authorization: Bearer <jeton>`.

Visiteurs. Le téléchargement (`POST /api/links/{linkId}/claim`) ne demande aucune identité. Un visiteur qui obtient un exemplaire reçoit un cookie `ticket` (signé, 2 min) qui lui permet de revenir chercher **le même** exemplaire sans en consommer un autre.

Garantie : nombre exact de téléchargements. Un lien créé avec `max = K` donne exactement K réponses 200 ou 202 porteuses d'un nouveau ticket, quel que soit le nombre de clics simultanés ; toutes les autres réponses aux clics sans ticket sont 410 (une fois le stock épuisé).

Mise en cache. Les réponses du clic portent toujours `Cache-Control: no-store`.

Portée. Ce fichier décrit toutes les routes HTTP. Le protocole temps réel (WebSocket `/collab` : connexion, contenu autorisé, confirmations `ack`, refus 4001, 4003, 4400 et 4500, versions, exemple de client) est décrit dans `docs/api-temps-reel.md`.

Parcours type pour un test. `POST /api/session` → `POST /api/documents` → écrire par `/collab` → `POST /api/documents/{docId}/links` → `POST /api/links/{linkId}/claim` (en rejouant le cookie `ticket` reçu jusqu'à obtenir 200).

Serveurs

- `https://{distribution}.cloudfront.net` — déploiement AWS (domaine CloudFront affiché par scripts/deploy.sh)
- `http://localhost:3000` — Développement local

Authentification

Nom	Mécanisme	Description
`admin`	en-tête `x-admin-token`	Jeton d'administration (SSM `/editeur/admin-token`), comparé à temps constant.
`session`	en-tête `Authorization: Bearer <jeton>`	Jeton renvoyé par `POST /api/session`.

Récapitulatif des routes

Méthode	Chemin	Rôle	Authentification
`GET`	`/sante`	Contrôle de santé (ALB, Auto Scaling)	aucune
`POST`	`/api/session`	Ouvrir une session de rédacteur	aucune
`GET`	`/api/documents`	Lister les documents dont on est éditeur	session
`POST`	`/api/documents`	Créer un document	session
`GET`	`/api/documents/{docId}`	Détail d'un document (éditeurs uniquement)	session
`POST`	`/api/documents/{docId}/title`	Renommer un document (éditeurs uniquement)	session
`GET`	`/api/documents/{docId}/versions`	Versions d'un document (50 plus récentes)	session
`GET`	`/api/me/credit`	Crédit d'écriture du jour (affichage)	session
`POST`	`/api/documents/{docId}/editors`	Partager un document avec un autre rédacteur	session
`GET`	`/api/documents/{docId}/links`	Liens du document (le lien actif d'abord, puis les plus récents)	session
`POST`	`/api/documents/{docId}/links`	Créer un lien de téléchargement	session
`POST`	`/api/links/{linkId}/close`	Fermer un lien avant épuisement (éditeurs du document)	session
`POST`	`/api/links/{linkId}/claim`	Cliquer sur un lien de téléchargement	aucune
`GET`	`/api/admin/documents-ouverts`	Documents ouverts (battement du serveur temps réel il y a moins de 90 s)	admin
`GET`	`/api/admin/liens-actifs`	Liens actifs et exemplaires restants (index creux liens-actifs)	admin
`GET`	`/api/admin/blocages`	Blocages en cours (rédacteurs 5 h, visiteurs 10 min), filtrés sur expireLe	admin

Santé

`GET /sante`

Contrôle de santé (ALB, Auto Scaling)

Ne dépend d'aucun service AWS.

Authentification : aucune.

Réponses

Code	Signification
200	Instance en état de servir.

Réponse 200

- en-tête `Cache-Control` : `no-store`

Champ	Type	Obligatoire	Contraintes et description
`statut`	`ok`	non	

```
{
  "statut": "ok"
}
```

Exemple d'appel :

```
curl -i -X GET 'https://xxxx.cloudfront.net/sante'
```

Session et crédit

`POST /api/session`

Ouvrir une session de rédacteur

L'e-mail est normalisé (minuscules, espaces retirés) et sert d'identifiant : crédit d'écriture, droits sur les documents et blocages lui sont rattachés.

Authentification : aucune.

Corps de la requête (`application/json`)

Champ	Type	Obligatoire	Contraintes et description
`name`	texte	oui	≥ 1 car., ≤ 80 car.
`email`	texte	oui	≥ 3 car., ≤ 254 car.

Exemple :

```
{
  "name": "Alan",
  "email": " Alan@Exemple.ORG"
}
```

Réponses

Code	Signification
200	Session ouverte.
400	Corps invalide ou adresse e-mail invalide.

Réponse 200

Champ	Type	Obligatoire	Contraintes et description
`token`	texte	oui	Jeton de session, valable 24 h.
`email`	texte	oui	E-mail normalisé.
`name`	texte	oui	

```
{
  "token": "eyJlbWFpbCI6...Rk0",
  "email": "alan@exemple.org",
  "name": "Alan"
}
```

Réponse 400

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "identite-invalide"
}
```

Exemple d'appel :

```
curl -i -X POST 'https://xxxx.cloudfront.net/api/session' \
  -H 'content-type: application/json' \
  -d '{"name": "Alan", "email": " Alan@Exemple.ORG"}'
```

`GET /api/me/credit`

Crédit d'écriture du jour (affichage)

Jour UTC. La garantie du crédit quotidien est assurée par le serveur temps réel, pas par cette route.

Authentification : `session`.

Réponses

Code	Signification
200	Crédit.
401	Jeton absent, falsifié ou expiré.

Réponse 200

Champ	Type	Obligatoire	Contraintes et description
`used`	entier	non	
`remaining`	entier	non	
`limit`	entier	non	

Réponse 401

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "session-invalide"
}
```

Exemple d'appel :

```
curl -i -X GET 'https://xxxx.cloudfront.net/api/me/credit' \
  -H 'Authorization: Bearer <jeton>'
```

Documents

`GET /api/documents`

Lister les documents dont on est éditeur

Triés par titre. Lecture filtrée de la table des documents (volume d'une association).

Authentification : `session`.

Réponses

Code	Signification
200	Documents.
401	Jeton absent, falsifié ou expiré.

Réponse 200

Champ	Type	Obligatoire	Contraintes et description
`documents`	liste de objet	non	
`documents[].docId`	texte (UUID)	—	
`documents[].title`	texte	—	
`documents[].ownerEmail`	texte	—	

Réponse 401

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "session-invalide"
}
```

Exemple d'appel :

```
curl -i -X GET 'https://xxxx.cloudfront.net/api/documents' \
  -H 'Authorization: Bearer <jeton>'
```

`POST /api/documents`

Créer un document

Le créateur en devient propriétaire et seul éditeur.

Authentification : `session`.

Corps de la requête (`application/json`)

Champ	Type	Obligatoire	Contraintes et description
`title`	texte	oui	≥ 1 car., ≤ 200 car.

Réponses

Code	Signification
201	Document créé.
400	Corps ou paramètres invalides (validation JSON Schema).
401	Jeton absent, falsifié ou expiré.

Réponse 201

Champ	Type	Obligatoire	Contraintes et description
`docId`	texte (UUID)	non	

Champ	Type	Obligatoire	Contraintes et description
`title`	texte	non	

Réponse 401

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "session-invalide"
}
```

Exemple d'appel :

```
curl -i -X POST 'https://xxxx.cloudfront.net/api/documents' \
  -H 'Authorization: Bearer <jeton>' \
  -H 'content-type: application/json' \
  -d '{}'
```

`GET /api/documents/{docId}`

Détail d'un document (éditeurs uniquement)

Authentification : `session`.

Paramètres

Nom	Où	Obligatoire	Type	Description
`docId`	chemin	oui	texte (UUID)	

Réponses

Code	Signification
200	Document.
401	Jeton absent, falsifié ou expiré.
403	Pas éditeur du document.
404	Document inconnu.

Réponse 200

Champ	Type	Obligatoire	Contraintes et description
`docId`	texte (UUID)	non	
`title`	texte	non	
`ownerEmail`	texte	non	

Champ	Type	Obligatoire	Contraintes et description
`editors`	liste de texte	non	
`seq`	entier	non	Dernière séquence acceptée du journal.

Réponse 401

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "session-invalide"
}
```

Exemple d'appel :

```
curl -i -X GET 'https://xxxx.cloudfront.net/api/documents/<docId>' \
  -H 'Authorization: Bearer <jeton>'
```

`POST /api/documents/{docId}/title`

Renommer un document (éditeurs uniquement)

Authentification : `session`.

Paramètres

Nom	Où	Obligatoire	Type	Description
`docId`	chemin	oui	texte (UUID)	

Corps de la requête (`application/json`)

Champ	Type	Obligatoire	Contraintes et description
`title`	texte	oui	≥ 1 car., ≤ 200 car.

Exemple :

```
{
  "title": "Journal — novembre 2026"
}
```

Réponses

Code	Signification
200	Titre enregistré (espaces de début et de fin retirés).
400	Titre vide ou corps invalide.
401	Jeton absent, falsifié ou expiré.

Code	Signification
403	Pas éditeur du document.
404	Document inconnu.

Réponse 200

Champ	Type	Obligatoire	Contraintes et description
`docId`	texte (UUID)	non	
`title`	texte	non	

Réponse 400

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "titre-invalide"
}
```

Réponse 401

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "session-invalide"
}
```

Exemple d'appel :

```
curl -i -X POST 'https://xxxx.cloudfront.net/api/documents/<docId>/title' \
  -H 'Authorization: Bearer <jeton>' \
  -H 'content-type: application/json' \
  -d '{"title": "Journal — novembre 2026"}'
```

`GET /api/documents/{docId}/versions`

Versions d'un document (50 plus récentes)

Instantanés de compaction (« automatique ») et versions nommées. L'enregistrement et la restauration passent par la connexion temps réel (messages `enregistrer` et `restaurer`) : une restauration est écrite dans le journal comme une modification, sans consommer de crédit.

Authentification : `session`.

Paramètres

Nom	Où	Obligatoire	Type	Description
`docId`	chemin	oui	texte (UUID)	

Réponses

Code	Signification
200	Versions, de la plus récente à la plus ancienne.
401	Jeton absent, falsifié ou expiré.
403	Pas éditeur du document.
404	Document inconnu.

Réponse 200

Champ	Type	Obligatoire	Contraintes et description
`versions`	liste de objet	non	
`versions[].createdAt`	texte (date ISO 8601)	—	
`versions[].seq`	entier	—	
`versions[].label`	texte	—	
`versions[].author`	texte	—	peut être null
`versions[].snapshotKey`	texte	—	

Réponse 401

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "session-invalide"
}
```

Exemple d'appel :

```
curl -i -X GET 'https://xxxx.cloudfront.net/api/documents/<docId>/versions' \
  -H 'Authorization: Bearer <jeton>'
```

`POST /api/documents/{docId}/editors`

Partager un document avec un autre rédacteur

Réservé aux éditeurs du document ; le nouvel éditeur a les mêmes droits (écrire, partager, créer des liens). Ajouter un éditeur déjà présent ne change rien.

Authentification : `session`.

Paramètres

Nom	Où	Obligatoire	Type	Description
`docId`	chemin	oui	texte (UUID)	

Corps de la requête (`application/json`)

Champ	Type	Obligatoire	Contraintes et description
`email`	texte	oui	≥ 3 car., ≤ 254 car.

Exemple :

```
{
  "email": "redaction@exemple.org"
}
```

Réponses

Code	Signification
200	Éditeur ajouté (e-mail normalisé).
400	Corps invalide ou adresse e-mail invalide.
401	Jeton absent, falsifié ou expiré.
403	Le rédacteur n'est pas éditeur du document.
404	Document inconnu.

Réponse 200

Champ	Type	Obligatoire	Contraintes et description
`docId`	texte (UUID)	non	
`email`	texte	non	

Réponse 400

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "identite-invalide"
}
```

Réponse 401

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "session-invalide"
}
```

Réponse 403

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "acces-refuse"
}
```

Réponse 404

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "document-inconnu"
}
```

Exemple d'appel :

```
curl -i -X POST 'https://xxxx.cloudfront.net/api/documents/<docId>/editors' \
  -H 'Authorization: Bearer <jeton>' \
  -H 'content-type: application/json' \
  -d '{"email": "redaction@exemple.org"}'
```

Liens et téléchargement

`GET /api/documents/{docId}/links`

Liens du document (le lien actif d'abord, puis les plus récents)

Affichage dans l'éditeur, rafraîchi toutes les 5 s (exemplaires restants).

Authentification : `session`.

Paramètres

Nom	Où	Obligatoire	Type	Description
`docId`	chemin	oui	texte (UUID)	

Réponses

Code	Signification
200	Liens.
401	Jeton absent, falsifié ou expiré.
403	Pas éditeur du document.
404	Document inconnu.

Réponse 200

Champ	Type	Obligatoire	Contraintes et description
`links`	liste de objet	non	
`links[].linkId`	texte (UUID)	—	
`links[].max`	entier	—	
`links[].remaining`	entier	—	Exemplaires encore disponibles (0 si épuisé ou fermé).
`links[].served`	entier	—	Exemplaires réservés par des visiteurs.
`links[].status`	`actif` | `epuise` | `ferme`	—	
`links[].opensAt`	texte (date ISO 8601)	—	peut être null
`links[].createdAt`	texte (date ISO 8601)	—	

Réponse 401

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "session-invalide"
}
```

Exemple d'appel :

```
curl -i -X GET 'https://xxxx.cloudfront.net/api/documents/<docId>/links' \
  -H 'Authorization: Bearer <jeton>'
```

`POST /api/documents/{docId}/links`

Créer un lien de téléchargement

Réservé aux éditeurs du document. Le lien donne toujours le document **tel qu'il est au moment du clic**, jamais tel qu'il était à la création du lien. `opensAt` (optionnel) : avant cette heure, le clic répond 425. Le PDF de l'état actuel est préparé dès la création : si personne n'écrit d'ici les premiers clics, ils reçoivent directement 200. **Un**

seul lien actif par document : tant qu'un lien a des exemplaires, une nouvelle création répond 409 avec ce lien ; il faut l'épuiser ou le fermer (`POST /api/links/{linkId}/close`). Deux créations simultanées : une seule réussit.

Authentification : `session`.

Paramètres

Nom	Où	Obligatoire	Type	Description
`docId`	chemin	oui	texte (UUID)	

Corps de la requête (`application/json`)

Champ	Type	Obligatoire	Contraintes et description
`max`	entier	oui	≥ 1, ≤ 100000 — Nombre de téléchargements autorisés.
`opensAt`	texte (date ISO 8601)	non	Heure d'ouverture (ISO 8601).

Exemple :

```
{
  "max": 500,
  "opensAt": "2026-11-01T18:00:00Z"
}
```

Réponses

Code	Signification
201	Lien créé.
400	Corps ou paramètres invalides (validation JSON Schema).
401	Jeton absent, falsifié ou expiré.
403	Le rédacteur n'est pas éditeur du document.
404	Document inconnu.
409	Le document a déjà un lien actif.

Réponse 201

Champ	Type	Obligatoire	Contraintes et description
`linkId`	texte (UUID)	non	
`max`	entier	non	
`opensAt`	texte (date ISO 8601)	non	peut être null
`claimPath`	texte	non	

Réponse 401

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "session-invalide"
}
```

Réponse 403

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "acces-refuse"
}
```

Réponse 404

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "document-inconnu"
}
```

Réponse 409

Champ	Type	Obligatoire	Contraintes et description
`erreur`	`lien-actif-existant`	non	
`lien`	objet	non	

Exemple d'appel :

```
curl -i -X POST 'https://xxxx.cloudfront.net/api/documents/<docId>/links' \
  -H 'Authorization: Bearer <jeton>' \
  -H 'content-type: application/json' \
  -d '{"max": 500, "opensAt": "2026-11-01T18:00:00Z"}'
```

`POST /api/links/{linkId}/close`

Fermer un lien avant épuisement (éditeurs du document)

Le stock est remis à 0 en une écriture atomique : les clics suivants reçoivent 410. Une réservation encore en attente du PDF (202) est honorée ; un exemplaire déjà remis ne se retélécharge plus avec le ticket (410). L'URL S3 déjà donnée reste valable jusqu'à son expiration (60 s). Les actions programmées du lien (pic autour de `opensAt`) sont retirées. Le document peut recevoir un nouveau lien. Idempotent : fermer un lien déjà fermé ou épuisé renvoie son état.

Authentification : `session`.

Paramètres

Nom	Où	Obligatoire	Type	Description
`linkId`	chemin	oui	texte (UUID)	

Réponses

Code	Signification
200	État du lien après fermeture.
401	Jeton absent, falsifié ou expiré.
403	Pas éditeur du document du lien.
404	Lien inconnu.

Réponse 200

Champ	Type	Obligatoire	Contraintes et description
`linkId`	texte (UUID)	non	
`max`	entier	non	
`remaining`	entier	non	Exemplaires encore disponibles (0 si épuisé ou fermé).
`served`	entier	non	Exemplaires réservés par des visiteurs.
`status`	`actif` | `epuise` | `ferme`	non	
`opensAt`	texte (date ISO 8601)	non	peut être null
`createdAt`	texte (date ISO 8601)	non	

Réponse 401

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

```
{
  "erreur": "session-invalide"
}
```

Exemple d'appel :

```
curl -i -X POST 'https://xxxx.cloudfront.net/api/links/<linkId>/close' \
  -H 'Authorization: Bearer <jeton>'
```

`POST /api/links/{linkId}/claim`

Cliquer sur un lien de téléchargement

Aucune identité demandée. Le visiteur sait toujours où il en est :

Code	`etat`	Signification	Que faire
200	`pret`	Exemplaire disponible	Télécharger `url` (valable 60 s)
202	`patienter`	Exemplaire **réservé**, PDF en préparation	Rappeler après `Retry-After` (le cookie `ticket` est renvoyé automatiquement)
425	`pas-ouvert`	Avant `opensAt`	Rappeler après `Retry-After`
410	`epuise`	Plus aucun exemplaire	Arrêter d'interroger
429	`bloque`	Trop de requêtes depuis cette adresse (blocage 10 min)	Rappeler après `Retry-After`
404	`inconnu`	Lien inexistant	—

Premier clic (sans ticket). Réserver = consommer : le stock est décrémenté de façon atomique, à condition qu'il reste au moins un exemplaire. En cas de succès, la réponse (200 ou 202) pose le cookie `ticket`.

Clics suivants (avec ticket). Le serveur relit la réservation : jamais de nouvel exemplaire consommé, jamais de nouveau cookie. Une réservation n'a qu'**une seule URL valide à la fois** : tant qu'elle n'a pas expiré, le même clic renvoie la même URL ; une fois expirée (ticket encore valide), une nouvelle URL la remplace, sur le **même** PDF. Deux retours simultanés obtiennent la même nouvelle URL. Le détenteur d'un ticket valide est servi même après l'épuisement du lien.

Contenu du PDF (à jour au moment du clic). Le PDF délivré contient au moins toutes les modifications acceptées avant le clic : le clic fixe `seqRequis` (séquence du document, lue de façon fortement cohérente) et l'URL n'est donnée que lorsqu'un PDF de séquence ≥ `seqRequis` est entièrement écrit. Ce PDF est un état cohérent du document (journal rejoué jusqu'à sa séquence), jamais un mélange. En attendant : 202, le PDF est généré (une génération sert toutes les demandes en attente).

Lien pas encore ouvert (425). Nous utilisons `425 Too Early` (RFC 8470) dans un sens élargi : le lien existe mais son heure d'ouverture (`opensAt`) n'est pas atteinte. Aucun exemplaire n'est consommé ; le client rappelle après `Retry-After` (entre 5 et 62 s, avec une part aléatoire de 0 à 2 s pour étaler les retours), et peut afficher `opensAt`.

Anti-abus (429). Au-delà de 30/N requêtes en 10 s depuis une même adresse (N = instances API), l'adresse est bloquée 10 min. Les rappels d'un visiteur qui détient un ticket valide pour ce lien ne sont pas comptés. Toute autre réponse (5xx pendant un pic) doit être réessayée par le client avec un délai croissant ; seul 404 arrête.

Limites assumées. Un exemplaire réservé puis abandonné n'est pas remis en stock. L'URL pré-signée est réutilisable pendant ses 60 s de validité.

Authentification : aucune.

Paramètres

Nom	Où	Obligatoire	Type	Description
`linkId`	chemin	oui	texte (UUID)	

Nom	Où	Obligatoire	Type	Description
`ticket`	cookie	non	texte	Ticket reçu au premier clic (signé, 2 min, `Path=/api/links/{linkId}`).

Réponses

Code	Signification
200	Exemplaire disponible.
202	Exemplaire réservé, PDF en préparation.
404	Lien inconnu ou identifiant mal formé.
410	Lien épuisé.
429	Anti-abus : plus de 30/N requêtes en 10 s depuis la même adresse (N = instances API ; adresse lue dans `CloudFront-Viewer-Address`). Blocage de 10 min, aucun exemplaire consommé.
425	Lien pas encore ouvert : `opensAt` est dans le futur. Aucun exemplaire consommé ; rappeler après `Retry-After` .

Réponse 200

- en-tête `Cache-Control` : `no-store`
- en-tête `Set-Cookie` : Présent seulement sur le premier clic qui réserve un exemplaire : `ticket=<jeton>; Max-Age=120; Path=/api/links/{linkId}; HttpOnly; Secure; SameSite=Lax` .

Champ	Type	Obligatoire	Contraintes et description
`etat`	`pret`	oui	
`url`	texte (URL)	oui	URL pré-signée S3 (GET), valable 60 s ; renouvelée sur le même PDF si elle a expiré et que le ticket est valide.
`expiresAt`	texte (date ISO 8601)	oui	

Réponse 202

- en-tête `Cache-Control` : `no-store`
- en-tête `Retry-After` : Délai conseillé avant de réinterroger, en secondes.
- en-tête `Set-Cookie` : Présent seulement sur le premier clic qui réserve un exemplaire : `ticket=<jeton>; Max-Age=120; Path=/api/links/{linkId}; HttpOnly; Secure; SameSite=Lax` .

Champ	Type	Obligatoire	Contraintes et description
`etat`	`patienter`	oui	
`retryAfter`	entier	oui	Secondes.

```
{
  "etat": "patienter",
  "retryAfter": 2
}
```

Réponse 404

- en-tête `Cache-Control` : `no-store`

Champ	Type	Obligatoire	Contraintes et description
`etat`	`pret` | `patienter` | `pas-ouvert` | `epuise` | `bloque` | `inconnu`	oui	

```
{
  "etat": "inconnu"
}
```

Réponse 410

- en-tête `Cache-Control` : `no-store`

Champ	Type	Obligatoire	Contraintes et description
`etat`	`pret` | `patienter` | `pas-ouvert` | `epuise` | `bloque` | `inconnu`	oui	

```
{
  "etat": "epuise"
}
```

Réponse 429

- en-tête `Cache-Control` : `no-store`
- en-tête `Retry-After` : Délai conseillé avant de réinterroger, en secondes.

Champ	Type	Obligatoire	Contraintes et description
`etat`	`bloque`	non	
`finLe`	texte (date ISO 8601)	non	
`retryAfter`	entier	non	

Réponse 425

- en-tête `Cache-Control` : `no-store`
- en-tête `Retry-After` : Délai conseillé avant de réinterroger, en secondes.

Champ	Type	Obligatoire	Contraintes et description
`etat`	`pas-ouvert`	oui	
`opensAt`	texte (date ISO 8601)	oui	
`retryAfter`	entier	oui	≥ 1, ≤ 60

Exemple d'appel :

```
curl -i -X POST 'https://xxxx.cloudfront.net/api/links/<linkId>/claim' \
  -b cookies.txt -c cookies.txt   # garde et renvoie le cookie ticket
```

Tableau de bord (admin)

`GET /api/admin/documents-ouverts`

Documents ouverts (battement du serveur temps réel il y a moins de 90 s)

Authentification : `admin`.

Réponses

Code	Signification
200	Documents ouverts, les plus fréquentés d'abord.
401	Jeton admin absent ou invalide.

Réponse 200

Champ	Type	Obligatoire	Contraintes et description
`documents`	liste de objet	non	
`documents[].docId`	texte (UUID)	—	
`documents[].title`	texte	—	
`documents[].connected`	entier	—	
`documents[].lastBeat`	texte (date ISO 8601)	—	

Exemple d'appel :

```
curl -i -X GET 'https://xxxx.cloudfront.net/api/admin/documents-ouverts' \
  -H 'x-admin-token: <jeton admin>'
```

`GET /api/admin/liens-actifs`

Liens actifs et exemplaires restants (index creux liens-actifs)

Authentification : `admin`.

Réponses

Code	Signification
200	Liens non épuisés.
401	Jeton admin absent ou invalide.

Réponse 200

Champ	Type	Obligatoire	Contraintes et description
`liens`	liste de objet	non	
`liens[].linkId`	texte (UUID)	—	
`liens[].docId`	texte (UUID)	—	
`liens[].creatorEmail`	texte	—	
`liens[].max`	entier	—	
`liens[].remaining`	entier	—	
`liens[].opensAt`	texte (date ISO 8601)	—	peut être null

Exemple d'appel :

```
curl -i -X GET 'https://xxxx.cloudfront.net/api/admin/liens-actifs' \
  -H 'x-admin-token: <jeton admin>'
```

`GET /api/admin/blocages`

Blocages en cours (rédacteurs 5 h, visiteurs 10 min), filtrés sur expireLe

Authentification : `admin`.

Réponses

Code	Signification
200	Blocages non expirés.
401	Jeton admin absent ou invalide.

Réponse 200

Champ	Type	Obligatoire	Contraintes et description
`blocages`	liste de objet	non	
`blocages[].subject`	texte	—	
`blocages[].reason`	texte	—	
`blocages[].expireLe`	texte (date ISO 8601)	—	

Exemple d'appel :

```
curl -i -X GET 'https://xxxx.cloudfront.net/api/admin/blocages' \
  -H 'x-admin-token: <jeton admin>'
```

Schémas communs

`Id`

texte (UUID), motif `^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$`

`Erreur`

Champ	Type	Obligatoire	Contraintes et description
`erreur`	texte	oui	

`Lien`

Champ	Type	Obligatoire	Contraintes et description
`linkId`	texte (UUID)	non	
`max`	entier	non	
`remaining`	entier	non	Exemplaires encore disponibles (0 si épuisé ou fermé).
`served`	entier	non	Exemplaires réservés par des visiteurs.
`status`	`actif` | `epuise` | `ferme`	non	
`opensAt`	texte (date ISO 8601)	non	peut être null
`createdAt`	texte (date ISO 8601)	non	

`Etat`

Champ	Type	Obligatoire	Contraintes et description
`etat`	`pret` | `patienter` | `pas-ouvert` | `epuise` | `bloque` | `inconnu`	oui	

Ce document complète `openapi.yaml`, qui décrit l'API HTTP. Il donne au jury de quoi écrire son propre client d'écriture, par exemple pour vérifier qu'aucune modification n'est perdue ou que le crédit quotidien est respecté.

1. En bref

Élément	Valeur
URL	`wss://<distribution>.cloudfront.net/collab` (en local : `ws://localhost:3000/collab`)
Protocole	Hocuspocus 4.7.1, c'est-à-dire la synchronisation Yjs (`y-protocols` sync et awareness) avec authentification par jeton
Client conseillé	`@hocuspocus/provider` **4.7.1** et `yjs` 13.6 ; en Node, ajouter `ws` comme implémentation WebSocket
Nom du document (`name`)	le `docId` renvoyé par `POST /api/documents`

Élément	Valeur
Jeton (`token`)	le `token` renvoyé par `POST /api/session` (valable 24 h)
Droit d'accès	l'adresse e-mail du jeton doit être **éditrice** du document, sinon l'authentification échoue
Contenu	un seul `Y.Text` , nommé **`contenu`**
Messages applicatifs	messages *stateless* Hocuspocus, en JSON (§4)

2. Règles à connaître

1. **Accepté = confirmé.** Une modification est acceptée quand elle est écrite dans DynamoDB. Le serveur ne l'applique à sa copie, ne la diffuse aux autres et ne la confirme qu'**après** cette écriture. La confirmation est le message `{"type":"ack","seq":N}` , où `N` est la séquence du journal qui la contient. Une modification qui n'a pas reçu d'`ack` n'est pas encore acceptée.
2. **Un `ack` par message de modification.** Hocuspocus traite les messages d'une connexion un par un. Chaque message de modification reçoit son `ack` dans l'ordre d'envoi. Plusieurs frappes groupées dans un même message (option `flushDelay` du provider, 200 ms dans l'éditeur) reçoivent donc un seul `ack` .
3. **Écriture au même endroit.** Yjs est un CRDT : deux rédacteurs peuvent insérer à la même position, toutes les copies convergent vers le même texte.
4. **Crédit quotidien.** Chaque rédacteur dispose de 10 000 caractères **insérés** par jour UTC, tous appareils et tous documents confondus. Les suppressions, la mise en forme et la restauration d'une version ne consomment rien. Le crédit restant se lit avec `GET /api/me/credit` .
5. **Contenu autorisé.** Uniquement du texte, avec ces attributs de mise en forme :

Attribut	Valeurs
`bold` , `italic` , `underline`	`true` (ou `null` / `false` pour retirer)
`font`	`"serif"` (Times) ou `"monospace"` (Courier) ; absent = sans empattement (Helvetica)
`size`	`"small"` , `"large"` ou `"huge"` ; absent = normale

   Tout autre contenu (autre type Yjs, objet, image, lien, couleur, taille libre…) est **refusé** avec le code 4400.
6. **Après un refus**, le serveur ferme la connexion au document. Avant toute reconnexion, le client **doit détruire son `Y.Doc`** et en créer un neuf : l'ancien contient la modification refusée, qui repartirait à la reconnexion.

3. Refus

Le serveur envoie d'abord un message `refus` , puis ferme la connexion au document. Hocuspocus 4 ne transmet pas de code WebSocket propre au document (une socket peut porter plusieurs documents). Le code se lit donc dans le message `refus` , et il est aussi repris au début de la raison de fermeture (par exemple `"4001 credit-epuise"`).

Code	`raison`	Cause	Ce que fait l'éditeur
4001	`credit-epuise`	La modification ferait dépasser 10 000 caractères aujourd'hui	Nouveau `Y.Doc`, reconnexion en **lecture seule** jusqu'à `finLe` (minuit UTC)
4003	`bloque`	5 refus en 1 minute : rédacteur bloqué 5 h	Nouveau `Y.Doc`, **aucune reconnexion** avant `finLe`
4400	`contenu-invalide`	Contenu ou attribut non autorisé (§2.5)	Nouveau `Y.Doc`, reconnexion
4500	`erreur-serveur`	Écriture DynamoDB impossible : rien n'a été accepté	Nouveau `Y.Doc`, reconnexion

Les refus 4001 et 4003 comptent pour le blocage. Un rédacteur bloqué peut toujours lire le document : il reçoit les modifications des autres.

4. Messages *stateless* (JSON)

Envoyés avec `provider.sendStateless(JSON.stringify(message))`, reçus dans le rappel `onStateless({ payload })`. Les types exacts sont `ServerMessage` et `ClientMessage` dans `packages/shared/src/realtime.ts`.

Du serveur vers le client

Message	Signification
`{"type":"ack","seq":42}`	Le message de modification précédent est écrit dans le journal à la séquence 42
`{"type":"refus","code":4001,"raison":"credit-epuise","finLe":1793059200000}`	Refus (§3) ; `finLe` en millisecondes depuis l'époque Unix, pour 4001 et 4003
`{"type":"version-enregistree","seq":42,"label":"Avant relecture"}`	Version nommée enregistrée (instantané immédiat)
`{"type":"restauree","seq":43}`	Version restaurée, écrite dans le journal à la séquence 43 comme une nouvelle modification
`{"type":"apercu","snapshotKey":"snapshots/<docId>/42.bin","delta":[{"insert":"Titre","attributes":{"bold":true}},{"insert":"\n"}]}`	Contenu d'une version, pour l'aperçu
`{"type":"erreur","message":"version inconnue"}`	Opération de version impossible

Du client vers le serveur

Message	Effet
`{"type":"enregistrer","label":"Avant relecture"}`	Enregistre la version actuelle (nom tronqué à 80 caractères ; vide = « Version sans nom »)
`{"type":"apercu","snapshotKey":"snapshots/<docId>/42.bin"}`	Demande le contenu d'une version (lecture seule)

Message	Effet
`{"type":"restaurer","snapshotKey":"snapshots/<docId>/42.bin"}`	Réapplique le contenu de cette version comme une nouvelle modification, sans crédit

Les clés `snapshotKey` viennent de `GET /api/documents/{docId}/versions`. Une clé qui n'appartient pas au document est refusée (`erreur`).

Présence et curseurs. L'éditeur publie dans l'*awareness* Yjs un champ `user = { name, email, color, colorLight }`, et y-quill y ajoute un champ `cursor` (positions relatives Yjs). Ces données ne passent pas par le journal et ne consomment pas de crédit.

5. Exemple de client minimal (Node 20)

```
// npm install @hocuspocus/provider@4.7.1 yjs ws
import { HocuspocusProvider, HocuspocusProviderWebsocket } from "@hocuspocus/provider";
import WebSocket from "ws";
import * as Y from "yjs";

const base = "https://xxxx.cloudfront.net";
const post = async (path, body, token) =>
  (await fetch(base + path, {
    method: "POST",
    headers: { "content-type": "application/json", ...(token ? { authorization: `Bearer ${token}` } :
        {}) },
    body: JSON.stringify(body),
  })).json();

const { token } = await post("/api/session", { name: "Jury", email: "jury@exemple.org" });
const { docId } = await post("/api/documents", { title: "Test du jury" }, token);

const doc = new Y.Doc();
const text = doc.getText("contenu");
const socket = new HocuspocusProviderWebsocket({ url: base.replace(/^http/, "ws") + "/collab",
        WebSocketPolyfill: WebSocket });
const provider = new HocuspocusProvider({
  websocketProvider: socket,
  name: docId,
  document: doc,
  token,
  onSynced: () => text.insert(0, "Bonjour"),              // une modification = un message
  onStateless: ({ payload }) => console.log(JSON.parse(payload)), // {"type":"ack","seq":1}
  onAuthenticationFailed: () => console.log("accès refusé"),
});
provider.attach(); // socket fournie par nous : le provider ne s'y attache pas tout seul
```

Pour vérifier la convergence : connecter plusieurs clients au même `docId`, les faire écrire au même endroit, attendre que chacun ait reçu ses `ack`, puis comparer `text.toString()` et `Y.encodeStateVector(doc)` entre les clients.

6. Clients et tests fournis

Fichier ou commande	Rôle
`tests/helpers/realtime.ts` (`TestWriter`)	Client de test complet : rattache chaque frappe à son `ack`, mesure le retard de confirmation, garde le refus et la raison de fermeture
`npm run test:c2 -- --base-url https://xxxx.cloudfront.net --bucket <BucketName>`	20 clients écrivent au même endroit à 8 caractères/s ; vérifie la convergence et que chaque `seq` confirmé est en base
`npm run test:c4 -- --base-url https://xxxx.cloudfront.net --bucket <BucketName>`	Un même e-mail sur 3 appareils et 2 documents : jamais plus de 10 000 caractères acceptés
`packages/realtime/test/*.test.ts`	Tests Vitest du protocole : refus, formats, versions, resynchronisation

`<BucketName>` est la sortie `BucketName` de la stack `EditeurPartage` (voir le README). L'option `--bucket` sert à relire l'état depuis la base (instantané S3 et journal DynamoDB) ; elle demande des identifiants AWS en lecture.